

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
df = pd.read_csv("professional_sales_dataset.csv")
df.head()
```

	Date	Region	Product	Customer_Segment	Sales_Units	Unit_Price	Discount	Shipping_Cost	Profit_Margin	Returns	Revenue
0	2023-04-13	East	Desktop	Consumer	8.0	NaN	0.10	7.34	0.11	0	4705.63
1	2023-12-15	East	Monitor	Home Office	11.0	934.22	0.05	5.58	0.12	0	9762.60
2	2023-09-28	North	Desktop	Consumer	8.0	965.85	NaN	40.80	0.19	0	6567.78
3	2023-04-17	South	Smartphone	Consumer	9.0	NaN	0.05	26.47	0.15	0	4771.41
4	2023-03-13	South	Laptop	Corporate	8.0	507.54	0.15	10.89	0.20	0	3451.27

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
print("\nData info:\n",df.info)
print("\nN.o of missing values in each column:\n",df.isnull().sum())
print("\nSummary Statistics for both Numeric and category columns:\n",df.describe(include='all'))
```

```
999  2023-11-29  West      Monitor      Consumer      15.0      425.59
      Discount  Shipping_Cost  Profit_Margin  Returns  Revenue
0      0.10      7.34      0.11      0  4705.63
1      0.05      5.58      0.12      0  9762.60
2      NaN      40.80      0.19      0  6567.78
3      0.05      26.47      0.15      0  4771.41
4      0.15      10.89      0.20      0  3451.27
..      ...      ...      ...      ...      ...
995      0.00      37.38      0.18      0  4824.27
996      0.10      43.46      0.25      0  5813.24
997      0.05      16.56      0.11      1  5446.82
998      0.00      5.94      0.12      0  7451.28
999      0.05      15.04      0.14      0  6064.66
```

```
[1000 rows x 11 columns]>
```

```
N.o of missing values in each column:
```

```
Date      0
Region     0
Product    0
Customer_Segment  0
Sales_Units  50
Unit_Price  50
Discount    50
Shipping_Cost  50
Profit_Margin  0
Returns     0
Revenue     0
dtype: int64
```

```
Summary Statistics for both Numeric and category columns:
```

```
      Date Region Product Customer_Segment Sales_Units Unit_Price \
count      1000      1000      1000      1000  950.000000  950.000000
unique       336         4         5         3         NaN         NaN
top  2023-04-09  North  Monitor      Consumer         NaN         NaN
freq         9       266       222       351         NaN         NaN
mean         NaN        NaN        NaN        NaN    9.896842  546.053884
```

std	0.056471	12.934306	0.056812	0.18132	3059.908054
min	0.000000	5.010000	0.100000	0.000000	431.640000
25%	0.000000	15.612500	0.150000	0.000000	2557.425000
50%	0.050000	26.830000	0.190000	0.000000	4568.200000
75%	0.100000	37.410000	0.240000	0.000000	6946.610000
max	0.150000	49.950000	0.300000	1.000000	17551.440000

```
df["Sales_Units"].fillna(df["Sales_Units"].median(), inplace=True)
```

/tmp/ipython-input-1665812672.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are operating is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col].method(value, inplace=True)

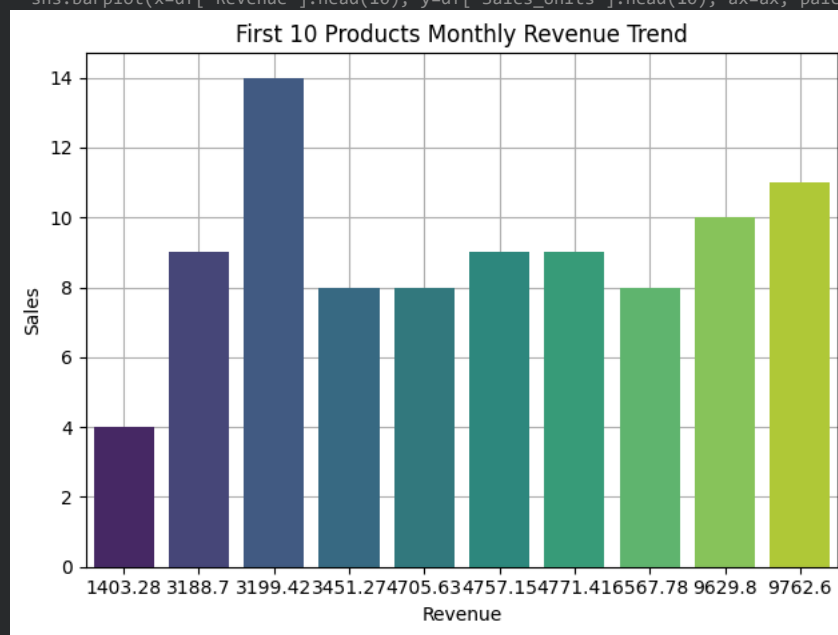
```
df["Sales_Units"].fillna(df["Sales_Units"].median(), inplace=True)
```

```
df["Date"]=pd.to_datetime(df["Date"])
df["Month"]=df["Date"].dt.to_period('M').astype(str)
```

```
fig,ax = plt.subplots()
sns.barplot(x=df["Revenue"].head(10), y=df["Sales_Units"].head(10), ax=ax, palette="viridis")
ax.set_xlabel("Revenue")
ax.set_ylabel("Sales")
ax.set_title("First 10 Products Monthly Revenue Trend")
plt.grid()
ax.set_axisbelow(True)
plt.tight_layout()
plt.figure()
plt.show()
```

/tmp/ipython-input-1917309174.py:2: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and use the 'palette' argument to map the data to a color palette.
sns.barplot(x=df["Revenue"].head(10), y=df["Sales_Units"].head(10), ax=ax, palette="viridis")



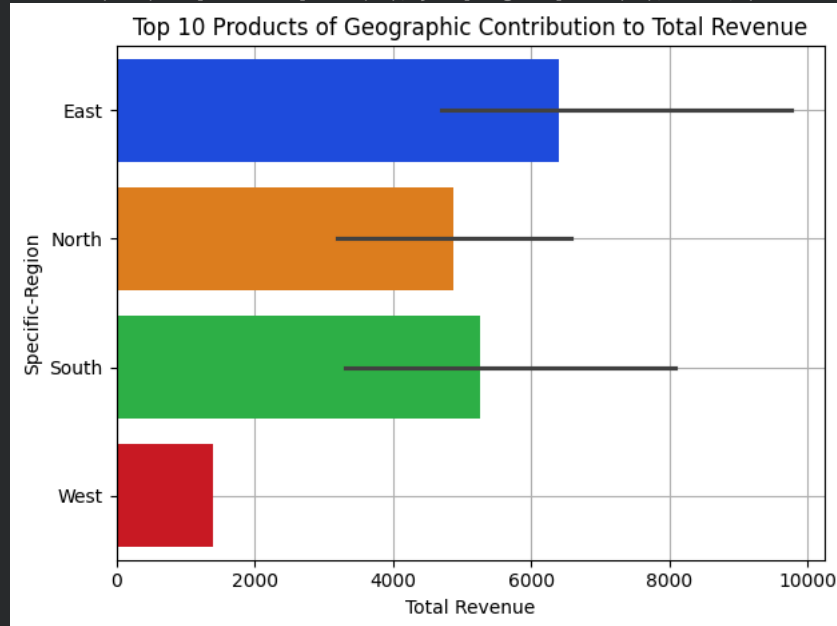
<Figure size 640x480 with 0 Axes>

```
fig,ax = plt.subplots()
sns.barplot(x=df['Revenue'].head(10), y=df['Region'].head(10), ax=ax, palette="bright")
ax.set_xlabel('Total Revenue')
ax.set_ylabel('Specific-Region')
ax.set_title('Top 10 Products of Geographic Contribution to Total Revenue')
plt.grid()
ax.set_axisbelow(True)
plt.tight_layout()
plt.figure()
plt.show()
```

/tmp/ipython-input-2640608903.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and

```
sns.barplot(x=df['Revenue'].head(10), y=df['Region'].head(10), ax=ax, palette="bright")
```



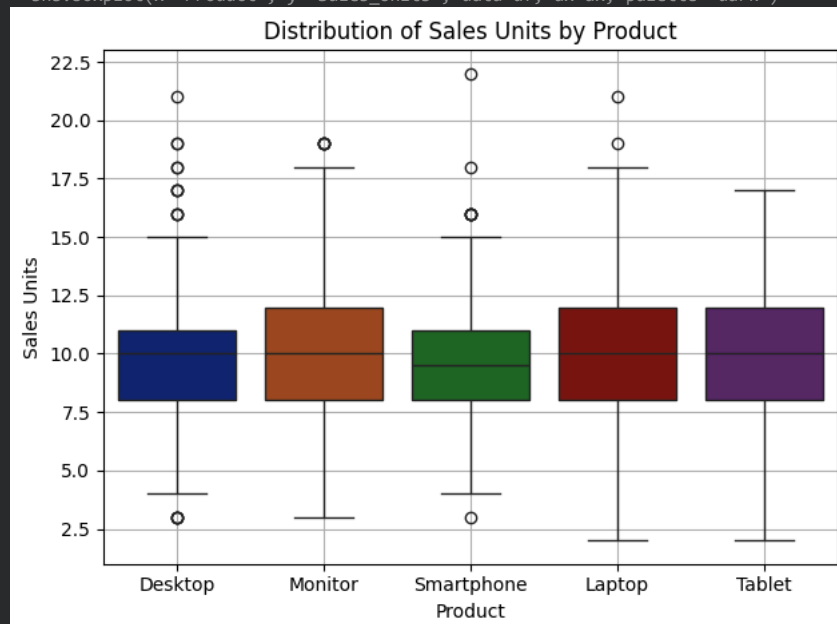
<Figure size 640x480 with 0 Axes>

```
fig,ax = plt.subplots()
sns.boxplot(x='Product', y='Sales_Units', data=df, ax=ax, palette="dark")
ax.set_xlabel('Product')
ax.set_ylabel('Sales Units')
ax.set_title('Distribution of Sales Units by Product')
plt.grid()
ax.set_axisbelow(True)
plt.tight_layout()
plt.figure()
plt.show()
```

/tmp/ipython-input-3236125047.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.boxplot(x='Product', y='Sales_Units', data=df, ax=ax, palette="dark")
```



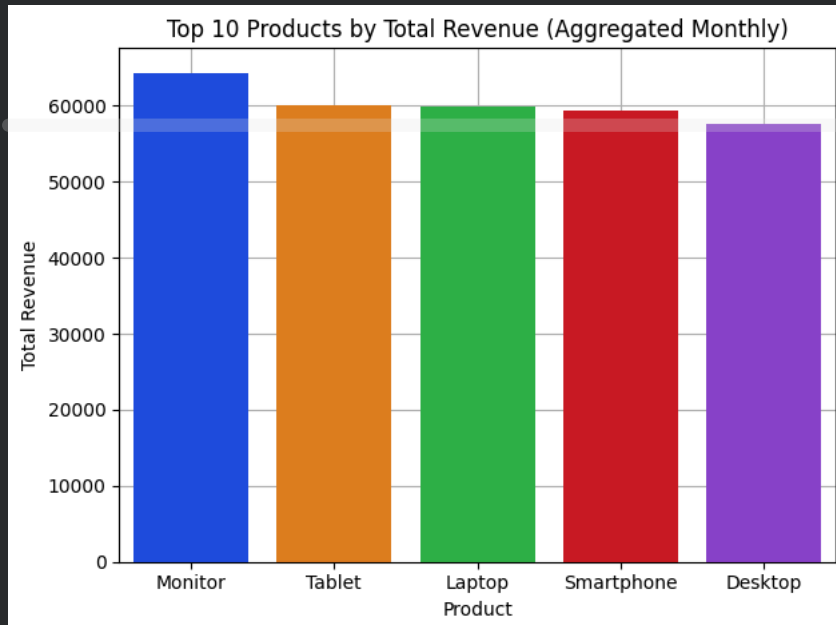
<Figure size 640x480 with 0 Axes>

```
fig,ax = plt.subplots()
axs = df.pivot_table(index="Month", columns="Product", values="Revenue")

# Calculate total revenue per product from the 'axs' pivot table
total_revenue_by_product = axs.sum().sort_values(ascending=False).reset_index()
```

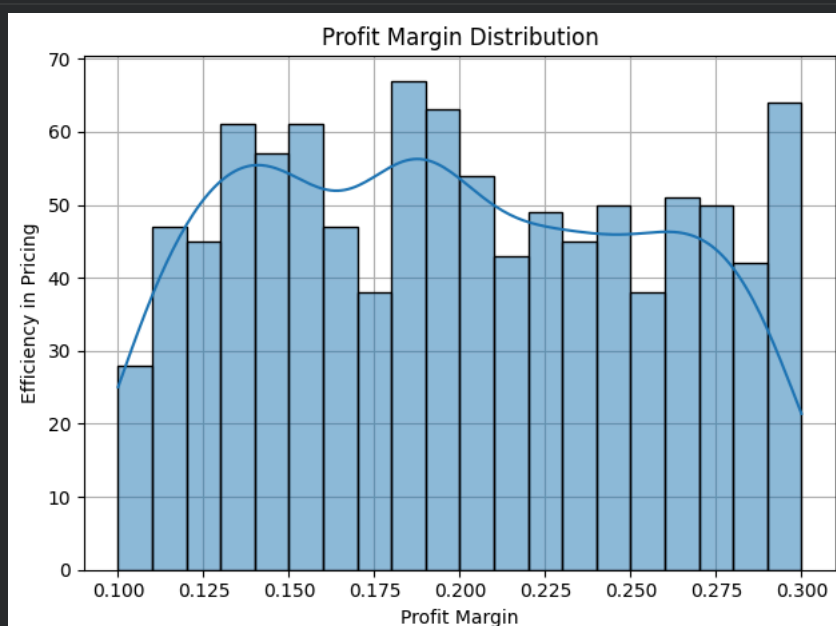
```
total_revenue_by_product.columns = ['Product', 'Total Revenue']
```

```
# Plot the top 10 products by total revenue
sns.barpplot(x='Product', y='Total Revenue', hue='Product', data=total_revenue_by_product.head(10), ax=ax, palette="bright",
ax.set_xlabel('Product')
ax.set_ylabel('Total Revenue')
ax.set_title('Top 10 Products by Total Revenue (Aggregated Monthly)')
plt.grid()
ax.set_axisbelow(True)
plt.tight_layout()
plt.figure()
plt.show()
```



<Figure size 640x480 with 0 Axes>

```
fig,ax = plt.subplots()
sns.histplot(df["Profit_Margin"], kde=True, bins=20)
plt.xlabel("Profit Margin")
plt.ylabel("Efficiency in Pricing")
plt.title("Profit Margin Distribution")
plt.grid()
ax.set_axisbelow(True)
plt.tight_layout()
plt.figure()
plt.show()
```



<Figure size 640x480 with 0 Axes>

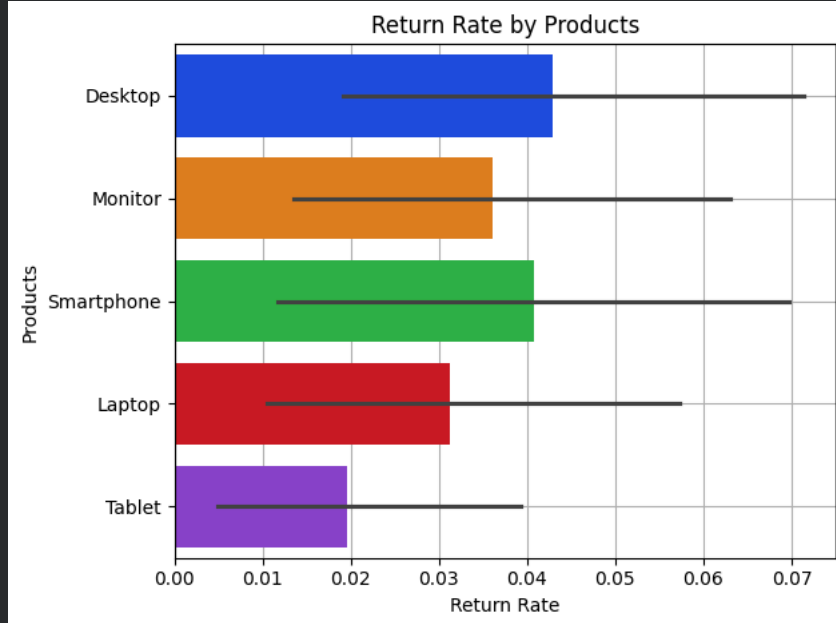
```
fig,ax = plt.subplots()
sns.barpplot(x=df['Returns'], y=df['Product'], ax=ax, palette="bright")
```

```
ax.set_xlabel('Return Rate')
ax.set_ylabel('Products')
ax.set_title('Return Rate by Products')
plt.grid()
ax.set_axisbelow(True)
plt.tight_layout()
plt.figure()
plt.show()
```

/tmp/ipython-input-2491896883.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and

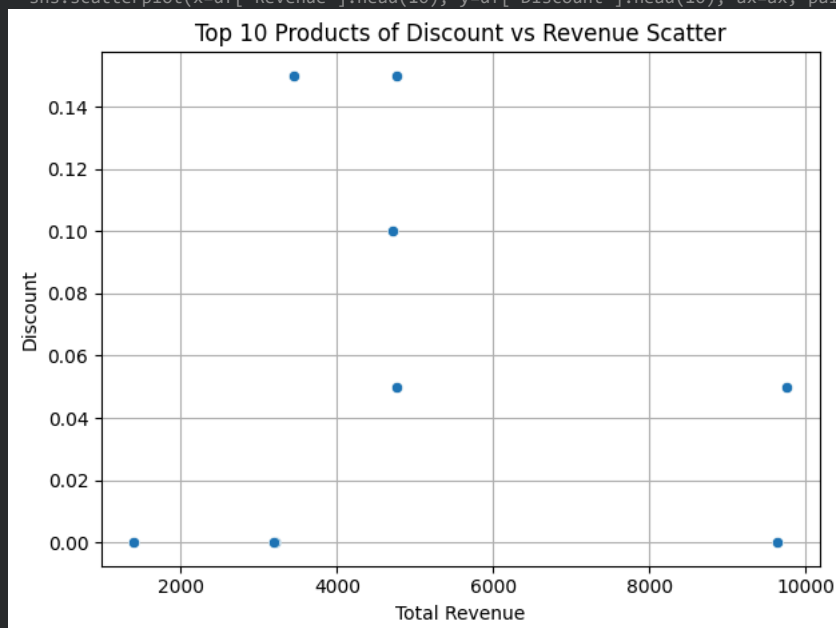
```
sns.barplot(x=df['Returns'], y=df['Product'], ax=ax, palette="bright")
```



<Figure size 640x480 with 0 Axes>

```
fig,ax = plt.subplots()
sns.scatterplot(x=df['Revenue'].head(10), y=df['Discount'].head(10), ax=ax, palette="bright")
ax.set_xlabel('Total Revenue')
ax.set_ylabel('Discount')
ax.set_title('Top 10 Products of Discount vs Revenue Scatter')
plt.grid()
ax.set_axisbelow(True)
plt.tight_layout()
plt.figure()
plt.show()
```

/tmp/ipython-input-16892125.py:2: UserWarning: Ignoring `palette` because no `hue` variable has been assigned.
sns.scatterplot(x=df['Revenue'].head(10), y=df['Discount'].head(10), ax=ax, palette="bright")



<Figure size 640x480 with 0 Axes>

Start coding or generate with AI

```
top_dates=df.groupby("Date")["Revenue"].sum().sort_values(ascending=False).head(5)
```

```
df.to_csv("Cleared_sales_dataset.csv",index=False)
```

```
print("\n Name: SanjayKumaran.D\n College: Vel Tech Multi tech Dr.Rangarajan Dr.Sakunthala Engineering College\n Email: sanj
```

```
Name: SanjayKumaran.D  
College: Vel Tech Multi tech Dr.Rangarajan Dr.Sakunthala Engineering College  
Email: sanjaykumaran6907@gmail.com
```